Linear regression and regularisation: least squares, ridge and lasso

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Linear regression is the simplest model that is learned by empirical risk minimisation, and the affine map it fits is the one computed inside every neuron of a standard neural network. This note derives the least-squares solution step by step — first for a line through a handful of points, then in matrix form — and explains its geometry, its algorithms and their cost. It shows when least squares becomes unstable and introduces the two classical regularisers, ridge (L_2) and lasso (L_1), with their solutions, their costs and their Bayesian reading, each with a small example that can be redone by hand. Linear probes of language-model activations and the sparse discovery of physical laws illustrate how much this small model still does.

1 The model

A linear (strictly, affine) regressor predicts

$$f_\theta(\mathbf{x}) = \mathbf{w}^\mathsf{T}\mathbf{x} + b = w_1 x_1 + \cdots + w_d x_d + b, \qquad \theta = (\mathbf{w}, b),\ \mathbf{w} \in \mathbb{R}^d,\ b \in \mathbb{R}.$$

Each weight is a sensitivity, $w_j = \partial f / \partial x_j$: the change in the prediction when x_j grows by one unit with the other inputs held fixed. The offset b is the prediction at $\mathbf{x} = \mathbf{0}$; a neural network would call this model one neuron with no activation function, and b its bias. We use the notation of Note 1, Section 2: n training pairs $(\mathbf{x}_i, y_i)$, the inputs stacked as the rows of the $n \times d$ design matrix X, the targets collected in $\mathbf{y} \in \mathbb{R}^n$.

1.1 The bias trick

The bias complicates every formula by a separate term. It is a good habit to absorb it by prepending a constant feature: with

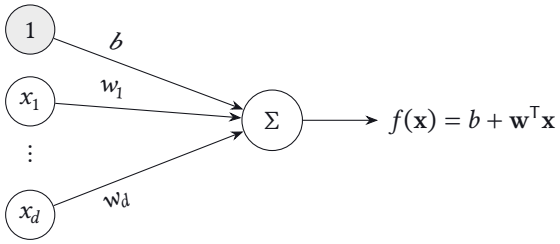
$$\tilde{\mathbf{x}} = (x_0, x_1, \ldots, x_d)^\mathsf{T} \in \mathbb{R}^{d+1}, \qquad \tilde{\mathbf{w}} = (w_0, w_1, \ldots, w_d)^\mathsf{T} \in \mathbb{R}^{d+1}, \qquad x_0 = 1,\ w_0 = b,$$

so that index 0 always refers to the bias, and we have $\tilde{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}} = b \cdot 1 + w_1 x_1 + \cdots + w_d x_d = f(\mathbf{x})$: an affine function of $\mathbf{x}$ is a linear function of $\tilde{\mathbf{x}}$. Stacking the $\tilde{\mathbf{x}}_i^\mathsf{T}$ as rows gives the $n \times (d+1)$ matrix $\tilde{X} = [\,\mathbf{1} \;\; X\,]$, the design matrix with a column of ones in front, and all n predictions at once are the matrix–vector product $\tilde{X}\tilde{\mathbf{w}}$. For the five cart positions of the first worked example in Section 2.1, with inputs $x = 0, 1, 2, 3, 4$,

$$\tilde{X} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 \end{pmatrix}^\mathsf{T}, \qquad \tilde{X}\begin{pmatrix} b \\ w \end{pmatrix} = \begin{pmatrix} b \\ b + w \\ b + 2w \\ b + 3w \\ b + 4w \end{pmatrix}.$$

To lighten the formulas we drop the tilde on $\tilde{\mathbf{w}}$: whenever $\mathbf{w}$ multiplies $\tilde{X}$ or $\tilde{\mathbf{x}}$ it is the full $(d+1)$-vector $(b, w_1, \dots, w_d)$; from Section 2.3 on, where the bias is split off again, $\mathbf{w}$ multiplying X or $\mathbf{x}$ means the d weights alone. The price of the trick is that the bias is now an ordinary weight; when we want to treat it differently — not penalising it in ridge regression, Section 4 — we have to say so explicitly.

The neuron picture. Drawn as a network (below), the model has one input node per feature plus a node that always outputs 1, and a single output node that forms the weighted sum of its inputs. The weights label the edges (the "synaptic strengths" of the neural vocabulary), and the bias is simply the weight on the constant node — the bias trick in pictures. The perceptron and the logistic neuron of Note 9 add a non-linear output to the same picture, and the networks of week 5 stack many such units.



Linear in the parameters. The model is linear in its *parameters*, not necessarily in the original measurements: $\mathbf{x}$ may contain logarithms, products or powers of raw quantities. Polynomial regression (the polynomial fits of Note 4, Section 5) is linear regression on the features $(x, x^2, \dots, x^p)$, and everything in this note applies to it unchanged; Note 10, Section 2, develops such feature maps systematically, and its later sections show how kernels make them implicit. Categorical inputs enter through the one-hot codes of Note 1, Section 3: the weight of each category is then the shift of the prediction for examples in that category. With a bias column, one category must be dropped (or a ridge penalty used), because the K one-hot columns of an attribute sum to the column of ones and make $\tilde{X}^\mathsf{T}\tilde{X}$ singular (Section 3). Transforming the *target* also changes the noise model: least squares on $\log y$ assumes additive Gaussian noise of constant size on $\log y$, that is, a roughly constant *relative* error on y itself.

2 Least squares

In the language of Note 4, Sections 1 and 3, a learning algorithm needs a family of models and a loss; linear regression takes the affine family of Section 1 and the squared loss of Note 1, Table 2, and empirical risk minimisation then means minimising the training loss. With the squared loss the empirical risk (up to the factor $1/n$) is

$$J(\mathbf{w}) = \sum_{i=1}^{n} (\mathbf{w}^\mathsf{T}\tilde{\mathbf{x}}_i - y_i)^2 = \left\| \tilde{X}\mathbf{w} - \mathbf{y} \right\|^2 .$$

Minimising it is the method of *least squares*. We first solve it for a single input with nothing but ordinary derivatives, then in matrix form for any d.

2.1 A line through points

For one input the objective is a function of two numbers,

$$J(b, w) = \sum_{i=1}^{n} (y_i - b - w x_i)^2.$$

It is a quadratic bowl in (b, w), so its minimum is where both partial derivatives vanish. By the chain rule,

$$\frac{\partial J}{\partial b} = -2 \sum_{i=1}^{n} (y_i - b - w x_i) = 0 \quad \Longrightarrow \quad n\bar{y} - nb - nw\bar{x} = 0 \quad \Longrightarrow \quad b = \bar{y} - w\bar{x},$$

$$\frac{\partial J}{\partial w} = -2 \sum_{i=1}^{n} x_i (y_i - b - w x_i) = 0,$$

with $\bar{x} = \frac{1}{n}\sum_i x_i$ and $\bar{y} = \frac{1}{n}\sum_i y_i$. The first condition says that the best line passes through the centre of mass $(\bar{x}, \bar{y})$ of the data. Substituting $b = \bar{y} - w\bar{x}$ into the second condition gives $\sum_i x_i\big[(y_i - \bar{y}) - w(x_i - \bar{x})\big] = 0$. The bracket sums to zero over i, because $\sum_i (y_i - \bar{y}) = \sum_i (x_i - \bar{x}) = 0$; hence $\bar{x}$ times the bracket also sums to zero. Subtracting this zero sum from the condition replaces the factor x_i by $x_i - \bar{x}$:

$$\sum_i (x_i - \bar{x})\big[(y_i - \bar{y}) - w(x_i - \bar{x})\big] = S_{xy} - w\, S_{xx} = 0, \qquad S_{xy} = \sum_i (x_i - \bar{x})(y_i - \bar{y}), \quad S_{xx} = \sum_i (x_i - \bar{x})^2,$$

and therefore

$$\hat{w} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}} = \frac{\widehat{\mathrm{Cov}}(x, y)}{\widehat{\mathrm{Var}}(x)}, \qquad \hat{b} = \bar{y} - \hat{w}\bar{x}. \tag{1}$$

The slope is the sample covariance of input and target divided by the sample variance of the input (whether both are normalised by $1/n$ or by $1/(n-1)$, the factor cancels in the ratio). A convenient measure of fit is the *coefficient of determination*

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2},$$

the fraction of the variance of the targets that the line explains (Section 2.4 shows why): $R^2 = 1$ for a perfect fit, $R^2 = 0$ for a line no better than the constant $\bar{y}$. It measures fit on the training data only; generalisation must still be checked on held-out data (Note 4, Section 4). On held-out data the same formula is used with the held-out targets and their mean; there it can even be negative, when the model predicts worse than that constant.

> **Worked example — fitting a line by hand**
>
> A cart on an air track is photographed once per second. Its positions (cm) at times $x = 0, 1, 2, 3, 4$ s are $y = 1, 3, 4, 7, 10$. Here $\bar{x} = 2$ and $\bar{y} = 5$; the table collects the centred values.

x_i	y_i	$x_i-\bar{x}$	$y_i-\bar{y}$	$(x_i-\bar{x})(y_i-\bar{y})$	$(x_i-\bar{x})^2$	$\hat{y}_i$	$y_i-\hat{y}_i$
0	1	−2	−4	8	4	0.6	0.4
1	3	−1	−2	2	1	2.8	0.2
2	4	0	−1	0	0	5.0	−1.0
3	7	1	2	2	1	7.2	−0.2
4	10	2	5	10	4	9.4	0.6
				$S_{xy}=22$	$S_{xx}=10$		sum $=0$

Hence $\hat{w}=22/10=2.2$ cm/s (the velocity) and $\hat{b}=5-2.2\times 2=0.6$ cm (the initial position). The residuals sum to zero, as the condition $\partial J/\partial b=0$ demands. The residual sum of squares is $0.16+0.04+1+0.04+0.36=1.6$ and the total sum of squares $\sum_i(y_i-\bar{y})^2=16+4+1+4+25=50$, so $R^2=1-1.6/50=0.968$. Figure 1 (left) shows the line and the residuals.

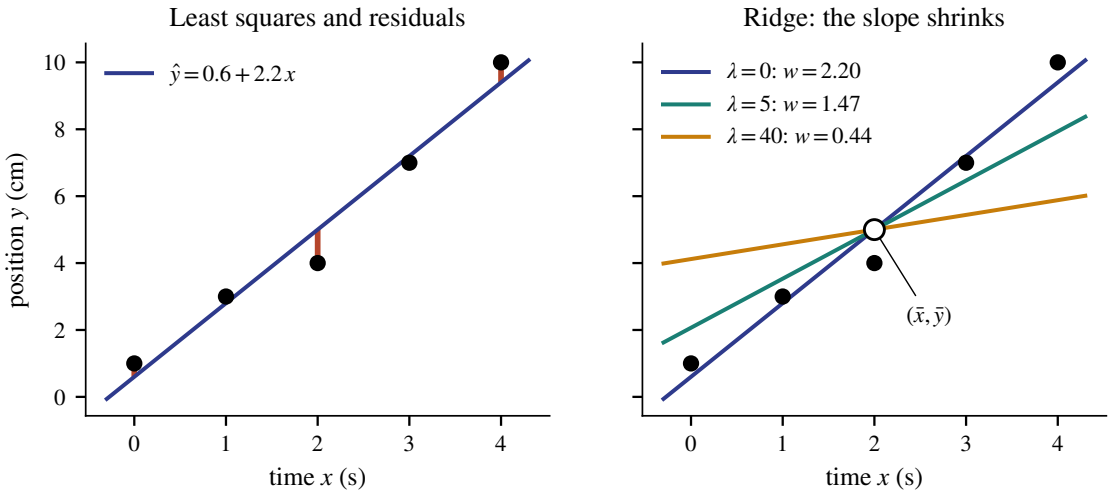


Figure 1. Left: the least-squares line of the cart example (Section 2.1); the vertical segments are the residuals whose squares are summed. Right: ridge regression on the same data (worked example in Section 4.1). As λ grows the slope shrinks towards zero; with the bias unpenalised, every line still passes through $(\bar{x},\bar{y})=(2,5)$.

Algorithm 1 Simple linear regression (one input)

Require: pairs (x_i,y_i), $i=1,\dots,n$
Ensure: intercept $\hat{b}$ and slope $\hat{w}$

1: $\bar{x}\leftarrow\frac{1}{n}\sum_i x_i,\quad \bar{y}\leftarrow\frac{1}{n}\sum_i y_i$ ▷ first pass
2: $S_{xy}\leftarrow\sum_i(x_i-\bar{x})(y_i-\bar{y}),\quad S_{xx}\leftarrow\sum_i(x_i-\bar{x})^2$ ▷ second pass
3: $\hat{w}\leftarrow S_{xy}/S_{xx}$ ▷ requires at least two distinct x_i
4: $\hat{b}\leftarrow\bar{y}-\hat{w}\bar{x}$
5: **return** $(\hat{b},\hat{w})$; predict with $\hat{y}=\hat{b}+\hat{w}x$

Running time — simple linear regression (Algorithm 1)

Training $O(n)$: two passes, constant work per point. Prediction $O(1)$ per query; memory $O(1)$. Centring first avoids the one-pass formula $S_{xy}=\sum x_iy_i-n\bar{x}\bar{y}$, which loses digits when the data sit far from 0.

2.2 The gradient in matrix form

For d inputs we repeat the calculation with vectors. Two rules of matrix calculus suffice. For a fixed vector $\mathbf{a}$ and a fixed symmetric matrix A,

$$\nabla_{\mathbf{w}}\,(\mathbf{a}^{\mathsf{T}}\mathbf{w}) = \mathbf{a}, \qquad \nabla_{\mathbf{w}}\,(\mathbf{w}^{\mathsf{T}}A\mathbf{w}) = 2A\mathbf{w}.$$

Both follow from partial derivatives: $\partial(\sum_j a_j w_j)/\partial w_k = a_k$, and $\partial(\sum_{i,j} w_i A_{ij} w_j)/\partial w_k = \sum_j A_{kj} w_j + \sum_i w_i A_{ik} = 2(A\mathbf{w})_k$ when $A = A^{\mathsf{T}}$. Now expand the square, using $(\tilde{X}\mathbf{w})^{\mathsf{T}}\mathbf{y} = \mathbf{y}^{\mathsf{T}}\tilde{X}\mathbf{w}$ (a scalar equals its transpose):

$$J(\mathbf{w}) = (\tilde{X}\mathbf{w} - \mathbf{y})^{\mathsf{T}}(\tilde{X}\mathbf{w} - \mathbf{y}) = \mathbf{w}^{\mathsf{T}}(\tilde{X}^{\mathsf{T}}\tilde{X})\mathbf{w} - 2\,(\tilde{X}^{\mathsf{T}}\mathbf{y})^{\mathsf{T}}\mathbf{w} + \mathbf{y}^{\mathsf{T}}\mathbf{y}.$$

The first term is a quadratic form with the symmetric matrix $A = \tilde{X}^{\mathsf{T}}\tilde{X}$, the second is linear with $\mathbf{a} = -2\tilde{X}^{\mathsf{T}}\mathbf{y}$, the third is constant. Hence

$$\nabla J(\mathbf{w}) = 2\tilde{X}^{\mathsf{T}}\tilde{X}\mathbf{w} - 2\tilde{X}^{\mathsf{T}}\mathbf{y} = 2\tilde{X}^{\mathsf{T}}(\tilde{X}\mathbf{w} - \mathbf{y}), \qquad H = \nabla^2 J = 2\tilde{X}^{\mathsf{T}}\tilde{X}.$$

The gradient has a direct reading: $\tilde{X}\mathbf{w}-\mathbf{y}$ is the vector of prediction errors, and $\tilde{X}^{\mathsf{T}}$ sums each feature column weighted by these errors. The Hessian is positive semi-definite, because $\mathbf{v}^{\mathsf{T}}\tilde{X}^{\mathsf{T}}\tilde{X}\mathbf{v} = \left\|\tilde{X}\mathbf{v}\right\|^2 \geq 0$ for every $\mathbf{v}$, so J is convex (Note 7, Section 4) and every stationary point is a global minimum. Setting the gradient to zero gives the *normal equations*

$$\tilde{X}^{\mathsf{T}}\tilde{X}\,\mathbf{w} = \tilde{X}^{\mathsf{T}}\mathbf{y}, \qquad \hat{\mathbf{w}} = (\tilde{X}^{\mathsf{T}}\tilde{X})^{-1}\tilde{X}^{\mathsf{T}}\mathbf{y} \quad \text{if } \tilde{X}^{\mathsf{T}}\tilde{X} \text{ is invertible.} \tag{2}$$

The $(d+1)\times(d+1)$ matrix $\tilde{X}^{\mathsf{T}}\tilde{X}$ contains the sums $\sum_i \tilde{x}_{ij}\tilde{x}_{ik}$ of products of features; $\tilde{X}^{\mathsf{T}}\mathbf{y}$ contains the sums $\sum_i \tilde{x}_{ij}y_i$. For $d = 1$, (2) reduces to the two conditions of Section 2.1. Note 2, Section 3, showed that minimising squared error is maximum likelihood under Gaussian noise, so $\hat{\mathbf{w}}$ is also the maximum-likelihood estimate of that model.

2.3 Centring removes the bias

Here, and wherever $\mathbf{w}$ multiplies X or $\mathbf{x}$ rather than $\tilde{X}$ or $\tilde{\mathbf{x}}$, $\mathbf{w} \in \mathbb{R}^d$ denotes the weights without the bias, and the full vector is $\tilde{\mathbf{w}} = (b, \mathbf{w})$ (Section 1.1). The first row of (2) belongs to the column of ones and reads $\sum_i (b + \mathbf{w}^{\mathsf{T}}\mathbf{x}_i - y_i) = 0$, i.e.

$$\hat{b} = \bar{y} - \hat{\mathbf{w}}^{\mathsf{T}}\bar{\mathbf{x}}, \qquad \bar{\mathbf{x}} = \tfrac{1}{n}\textstyle\sum_i \mathbf{x}_i,$$

exactly as for the line. Substituting it back, each residual becomes $y_i - b - \mathbf{w}^{\mathsf{T}}\mathbf{x}_i = (y_i - \bar{y}) - \mathbf{w}^{\mathsf{T}}(\mathbf{x}_i - \bar{\mathbf{x}})$. So the weights solve the least-squares problem for the *centred* data X_c (rows $\mathbf{x}_i - \bar{\mathbf{x}}$) and $\mathbf{y}_c$ (entries $y_i - \bar{y}$), with no bias at all:

$$X_c^{\mathsf{T}}X_c\,\hat{\mathbf{w}} = X_c^{\mathsf{T}}\mathbf{y}_c, \qquad \hat{b} = \bar{y} - \hat{\mathbf{w}}^{\mathsf{T}}\bar{\mathbf{x}}.$$

This is the second way of handling the bias: centre, solve a $d \times d$ system, and recover b at the end. It is the form we use for ridge and lasso, where the bias must not be penalised. When the features are already centred, the column of ones is orthogonal to the others, $\tilde{X}^{\mathsf{T}}\tilde{X}$ is block diagonal and $\hat{b} = \bar{y}$, as in the next example.

> **Worked example — normal equations with two features**
>
> The output y of a detector channel is recorded at five settings of two control voltages, given as deviations from their nominal values (so each column has mean zero). We write $\mathbf{f}_j \in \mathbb{R}^n$ for *column* j of X, the values of feature j over all n examples, to keep it apart from the example

$\mathbf{x}_i \in \mathbb{R}^d$, which is *row i*:

$$\mathbf{f}_1 = (-2,-1,0,1,2), \quad \mathbf{f}_2 = (1,-1,-1,0,1), \quad \mathbf{y} = (1,2,4,5,8).$$

The sums needed are $n = 5$, $\sum_i x_{i1} = \sum_i x_{i2} = 0$, $\sum_i y_i = 20$, $\|\mathbf{f}_1\|^2 = 10$, $\|\mathbf{f}_2\|^2 = 4$, $\mathbf{f}_1^\mathsf{T}\mathbf{f}_2 = -2+1+0+0+2 = 1$, $\mathbf{f}_1^\mathsf{T}\mathbf{y} = -2-2+0+5+16 = 17$ and $\mathbf{f}_2^\mathsf{T}\mathbf{y} = 1-2-4+0+8 = 3$. The normal equations (2) are

$$\begin{pmatrix} 5 & 0 & 0 \\ 0 & 10 & 1 \\ 0 & 1 & 4 \end{pmatrix} \begin{pmatrix} b \\ w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} 20 \\ 17 \\ 3 \end{pmatrix}.$$

The first row gives $b = 4 = \bar{y}$. The remaining 2×2 system has determinant $10 \cdot 4 - 1 \cdot 1 = 39$ and

$$\begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \frac{1}{39} \begin{pmatrix} 4 & -1 \\ -1 & 10 \end{pmatrix} \begin{pmatrix} 17 \\ 3 \end{pmatrix} = \frac{1}{39} \begin{pmatrix} 65 \\ 13 \end{pmatrix} = \begin{pmatrix} 5/3 \\ 1/3 \end{pmatrix} \approx \begin{pmatrix} 1.667 \\ 0.333 \end{pmatrix}.$$

The fitted values $\hat{y} = 4 + \frac{5}{3}x_1 + \frac{1}{3}x_2$ are $(1, 2, 3.667, 5.667, 7.667)$ and the residuals $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = (0, 0, \frac{1}{3}, -\frac{2}{3}, \frac{1}{3})$. Check: $\sum_i r_i = 0$, $\mathbf{f}_1^\mathsf{T}\mathbf{r} = -\frac{2}{3} + \frac{2}{3} = 0$ and $\mathbf{f}_2^\mathsf{T}\mathbf{r} = -\frac{1}{3} + \frac{1}{3} = 0$ — the residual is orthogonal to all three columns, as the next subsection explains. `np.linalg.lstsq` returns the same $(4, 1.667, 0.333)$.

2.4 Geometry: an orthogonal projection

The fitted values $\hat{\mathbf{y}} = \tilde{X}\hat{\mathbf{w}}$ lie in the column space of $\tilde{X}$, the set of all predictions the model can make (a subspace of $\mathbb{R}^n$ of dimension $d+1$ when the columns are independent, less otherwise). The normal equations say $\tilde{X}^\mathsf{T}(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$: the residual is orthogonal to every column, so $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto that space, the closest point to the observations. In matrix form $\hat{\mathbf{y}} = P\mathbf{y}$ with the projection ("hat") matrix $P = \tilde{X}(\tilde{X}^\mathsf{T}\tilde{X})^{-1}\tilde{X}^\mathsf{T}$, which satisfies $P^2 = P$: projecting twice changes nothing. Two consequences are worth remembering. With a constant column, the residuals sum to zero. And R^2 is a Pythagoras theorem: write the centred targets $\mathbf{y}_c = \mathbf{y} - \bar{y}\mathbf{1}$ as

$$\mathbf{y} - \bar{y}\mathbf{1} = (\hat{\mathbf{y}} - \bar{y}\mathbf{1}) + (\mathbf{y} - \hat{\mathbf{y}}).$$

The first vector lies in the column space (both $\hat{\mathbf{y}}$ and $\mathbf{1}$ do, since $\mathbf{1}$ is a column of $\tilde{X}$); the second is the residual, orthogonal to that space. The two are perpendicular, so $\|\mathbf{y}_c\|^2 = \|\hat{\mathbf{y}} - \bar{y}\mathbf{1}\|^2 + \|\mathbf{y} - \hat{\mathbf{y}}\|^2$: the total sum of squares equals the *explained* sum of squares (the spread of the fitted values about $\bar{y}$) plus the residual sum of squares. Hence $R^2 = 1 - \text{residual/total} = \text{explained/total}$ lies between 0 and 1 for least squares with a bias. In the detector example: $30 = 29.33 + 0.67$, so $R^2 = 0.978$.

2.5 Computing the solution

Normal equations, but not the inverse. Never form the inverse in (2). The matrix $\tilde{X}^\mathsf{T}\tilde{X}$ is symmetric and (if the columns are independent) positive definite, so the system is solved by a Cholesky factorisation $\tilde{X}^\mathsf{T}\tilde{X} = LL^\mathsf{T}$ and two triangular solves (Algorithm 2). This is the fastest exact method and can be accumulated in one pass over data that do not fit in memory, since $\tilde{X}^\mathsf{T}\tilde{X} = \sum_i \tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^\mathsf{T}$ and $\tilde{X}^\mathsf{T}\mathbf{y} = \sum_i y_i\tilde{\mathbf{x}}_i$.

QR or SVD. The weakness of the normal equations is numerical. Every $n \times p$ matrix ($n \geq p$) can be factorised as $\tilde{X} = USV^\mathsf{T}$, the (thin) *singular value decomposition*: $U \in \mathbb{R}^{n\times p}$ and $V \in \mathbb{R}^{p\times p}$ have orthonormal columns and S is diagonal with entries $s_1 \geq s_2 \geq \ldots \geq 0$, the *singular values*.

The s_j^2 are the eigenvalues of $\tilde{X}^\mathsf{T}\tilde{X}$ and the columns of V (the right singular vectors) its eigenvectors; Section 4.2 uses this in detail. The condition number κ of a matrix (ratio of largest to smallest singular value) measures how much relative errors are amplified; forming $\tilde{X}^\mathsf{T}\tilde{X}$ squares it, $\kappa(\tilde{X}^\mathsf{T}\tilde{X}) = \kappa(\tilde{X})^2$, and loses about twice as many significant digits. Factorising $\tilde{X} = QR$ with Q having orthonormal columns and R upper triangular avoids the squaring. Substituting, $\tilde{X}^\mathsf{T}\tilde{X} = R^\mathsf{T}Q^\mathsf{T}QR = R^\mathsf{T}R$ because $Q^\mathsf{T}Q = I$, and $\tilde{X}^\mathsf{T}\mathbf{y} = R^\mathsf{T}Q^\mathsf{T}\mathbf{y}$, so (2) becomes $R^\mathsf{T}R\mathbf{w} = R^\mathsf{T}Q^\mathsf{T}\mathbf{y}$. When the columns are independent, R^T is invertible and may be cancelled, leaving the triangular system $R\mathbf{w} = Q^\mathsf{T}\mathbf{y}$ (Algorithm 3). Library routines such as `np.linalg.lstsq` use the SVD instead (LAPACK's `gelsd`): substituting $\tilde{X} = USV^\mathsf{T}$ in the same way gives $\hat{\mathbf{w}} = V\,\mathrm{diag}(1/s_j)U^\mathsf{T}\mathbf{y}$, where singular values below a small tolerance are dropped rather than inverted (Algorithm 4). This costs a few times more than QR but also copes with rank-deficient $\tilde{X}$, where it returns the solution of smallest norm (Section 3).

Algorithm 2 Least squares by the normal equations (Cholesky)

Require: design matrix $\tilde{X} \in \mathbb{R}^{n\times p}$ (with the column of ones, $p = d + 1$), targets $\mathbf{y} \in \mathbb{R}^n$
Ensure: weights $\hat{\mathbf{w}} \in \mathbb{R}^p$

1: $G \leftarrow \tilde{X}^\mathsf{T}\tilde{X}$ ▷ $p \times p$ Gram matrix, $O(np^2)$
2: $\mathbf{c} \leftarrow \tilde{X}^\mathsf{T}\mathbf{y}$ ▷ $O(np)$
3: $L \leftarrow \textsc{Cholesky}(G)$ ▷ $G = LL^\mathsf{T}$, L lower triangular, $O(p^3)$
4: solve $L\mathbf{z} = \mathbf{c}$ by forward substitution, then $L^\mathsf{T}\hat{\mathbf{w}} = \mathbf{z}$ by back substitution ▷ $O(p^2)$
5: **return** $\hat{\mathbf{w}}$; predict with $\hat{y} = \hat{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}}$

Algorithm 3 Least squares by Householder QR factorisation

Require: $\tilde{X} \in \mathbb{R}^{n\times p}$ with independent columns, $\mathbf{y} \in \mathbb{R}^n$
Ensure: weights $\hat{\mathbf{w}}$

1: $(Q, R) \leftarrow \textsc{HouseholderQR}(\tilde{X})$ ▷ $Q \in \mathbb{R}^{n\times p}$, $Q^\mathsf{T}Q = I$, R upper triangular, $O(np^2)$
2: $\mathbf{c} \leftarrow Q^\mathsf{T}\mathbf{y}$ ▷ $O(np)$
3: solve $R\hat{\mathbf{w}} = \mathbf{c}$ by back substitution ▷ $O(p^2)$
4: **return** $\hat{\mathbf{w}}$ ▷ residual sum of squares: $\|\mathbf{y}\|^2 - \|\mathbf{c}\|^2$, see below

Algorithm 4 Least squares by the thin SVD (minimum-norm solution, as in `lstsq`)

Require: $\tilde{X} \in \mathbb{R}^{n\times p}$ (any rank), $\mathbf{y} \in \mathbb{R}^n$, relative tolerance τ (e.g. 10^{-12})
Ensure: the minimum-norm minimiser $\hat{\mathbf{w}}$ of $\|\tilde{X}\mathbf{w} - \mathbf{y}\|^2$

1: $(U, \mathbf{s}, V) \leftarrow \textsc{ThinSVD}(\tilde{X})$ ▷ $r = \min(n, p)$ singular values, $O(npr)$
2: $\mathbf{c} \leftarrow U^\mathsf{T}\mathbf{y}$ ▷ components of $\mathbf{y}$ along the $\mathbf{u}_j$, $O(nr)$
3: $\hat{\mathbf{w}} \leftarrow \sum_{j:\ s_j > \tau s_1} \mathbf{v}_j\, c_j/s_j$ ▷ tiny s_j are dropped, not inverted; $O(pr)$
4: **return** $\hat{\mathbf{w}}$

In Algorithm 3, the fitted values are $\hat{\mathbf{y}} = QQ^\mathsf{T}\mathbf{y}$, the projection of $\mathbf{y}$ onto the column space, and $\mathbf{c} = Q^\mathsf{T}\mathbf{y}$ holds its coordinates in the orthonormal basis Q; by Pythagoras $\|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \|\mathbf{y}\|^2 - \|\hat{\mathbf{y}}\|^2 = \|\mathbf{y}\|^2 - \|\mathbf{c}\|^2$, so the residual sum of squares comes for free. Algorithm 4 works also when $d + 1 > n$ or when columns are dependent: the dropped directions are exactly those along which the data say nothing, and leaving them at zero gives the solution of smallest norm.

Running time — least squares, direct methods (Algorithms 2, 3 and 4)

With n examples and $p = d + 1$ parameters. *Normal equations*: forming $\tilde{X}^\mathsf{T}\tilde{X}$ costs $O(np^2)$ (each of the p^2 entries is a sum over n products), $\tilde{X}^\mathsf{T}\mathbf{y}$ costs $O(np)$, Cholesky $O(p^3)$ (about $p^3/3$ operations), the triangular solves $O(p^2)$; total $O(np^2 + p^3)$, usually dominated by np^2 because $n > p$. Memory: $O(p^2)$ for

the Gram matrix, and the data may be streamed. *QR*: $O(np^2)$ time (about twice the normal equations), $O(np)$ memory since the whole matrix is factorised; the SVD has the same order, $O(np\min(n,p))$, with a constant a few times larger, and $O(np)$ memory. *Prediction*: $O(d)$ per query, one dot product. *Typical sizes*: $n = 10^6$, $d = 10^3$ means $np^2 \approx 10^{12}$ operations — seconds to minutes on a laptop, much less on a GPU, where these dense matrix products run in parallel. For $d \sim 10^5$ (text or genomic features) the p^2 memory (10^{10} numbers) and the p^3 time are prohibitive, and iterative methods take over.

Gradient descent. The normal equations are the exception, not the rule: they exist only because a linear model with the squared loss (plus, possibly, an L_2 penalty, Section 4) has a *quadratic* objective, whose gradient is linear in $\mathbf{w}$. With another loss (the absolute or Huber error, the logistic loss of Note 9) or a non-linear model, setting the gradient to zero gives equations that cannot be solved in closed form, and iterative minimisation (Note 7, Section 3) is the general method; the lasso of Section 5 is a first example. Even for least squares, gradient descent pays off for very large n or d, because it needs only matrix–vector products. It is customary to use the *mean* loss $L(\mathbf{w}) = \frac{1}{n}\left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 = J(\mathbf{w})/n$, whose size does not grow with the number of examples. Dividing by n does not move the minimiser; it only rescales the gradient, so the safe step size depends on which form is used. The update is

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla L(\mathbf{w}) = \mathbf{w} - \eta\,\frac{2}{n}\,\tilde{X}^{\mathsf{T}}(\tilde{X}\mathbf{w} - \mathbf{y}).$$

The Hessian of L is the constant matrix $\frac{2}{n}\tilde{X}^{\mathsf{T}}\tilde{X}$, so the analysis of Note 7, Section 3, applies exactly: the step must satisfy $\eta < 2/\mu_{\max}$, with $\mu_{\max}$ the largest eigenvalue of $\frac{2}{n}\tilde{X}^{\mathsf{T}}\tilde{X}$, and the number of iterations grows with its condition number — one more reason to standardise the features. Mini-batch gradients (Note 7, Section 5) replace the sum over all n examples by a sum over a random batch of n' and make the method work on data that do not fit in memory (Algorithm 5).

Worked example — one gradient-descent step

Take the cart data of Section 2.1 with $\theta = (b, w)$, start at $\theta_0 = (0, 0)$ and use $\eta = 0.05$, minimising the mean loss L. At the start all predictions are zero, so the errors are $\tilde{X}\theta_0 - \mathbf{y} = -\mathbf{y}$ and

$$\nabla L(\theta_0) = -\frac{2}{5}\,\tilde{X}^{\mathsf{T}}\mathbf{y} = -\frac{2}{5}\begin{pmatrix}\sum_i y_i \\ \sum_i x_i y_i\end{pmatrix} = -\frac{2}{5}\begin{pmatrix}25 \\ 72\end{pmatrix} = \begin{pmatrix}-10 \\ -28.8\end{pmatrix},$$

with $\sum_i x_i y_i = 0+3+8+21+40 = 72$. The step gives $\theta_1 = (0,0) - 0.05\,(-10, -28.8) = (0.5, 1.44)$, and the mean squared error falls from $L(\theta_0) = \frac{1}{5}(1 + 9 + 16 + 49 + 100) = 35$ to $L(\theta_1) = 4.10$; a second step gives $(0.662, 1.916)$ with $L = 0.74$. The minimum is $L(0.6, 2.2) = 1.6/5 = 0.32$. Is the step size safe? The Hessian $\frac{2}{5}\tilde{X}^{\mathsf{T}}\tilde{X} = \frac{2}{5}\left(\begin{smallmatrix}5 & 10\\ 10 & 30\end{smallmatrix}\right) = \left(\begin{smallmatrix}2 & 4\\ 4 & 12\end{smallmatrix}\right)$ has eigenvalues 0.60 and 13.40, so any $\eta < 2/13.40 = 0.149$ converges; with $\eta = 0.05$ the iterate agrees with $(0.600, 2.200)$ to three decimals after 200 steps. The condition number $13.40/0.60 \approx 22$ comes from the uncentred x: with $x - \bar{x}$ the Hessian is $\mathrm{diag}(2, 4)$.

A stochastic step. Stochastic gradient descent uses one example at a time, with the gradient $2(\theta^{\mathsf{T}}\tilde{\mathbf{x}}_i - y_i)\,\tilde{\mathbf{x}}_i$ of its own squared error. From $\theta_0 = (0,0)$ the example $(x, y) = (3, 7)$ gives $2(0 - 7)(1, 3) = (-14, -42)$ and the step $\theta_1 = (0.7, 2.1)$; the example $(0, 1)$ gives $(0.1, 0)$ and $(4, 10)$ gives $(1.0, 4.0)$. Each step costs a fifth of the batch step, but its direction depends on which example is drawn, and the mean squared error after it ranges from 0.35 to 34.0. Averaged over the five possible examples the step is $(0.5, 1.44)$, exactly the batch step: the stochastic gradient is unbiased but noisy (Note 7, Section 5).

Algorithm 5 Least squares by mini-batch stochastic gradient descent

Require: $\tilde{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y}$, learning rate η, batch size n', number of epochs E, penalty $\lambda' \geq 0$
Ensure: approximate minimiser $\mathbf{w}$ of $L(\mathbf{w}) + \lambda' \sum_{j \geq 1} w_j^2$, with $L(\mathbf{w}) = \frac{1}{n} \left\| \tilde{X}\mathbf{w} - \mathbf{y} \right\|^2$

$\mathbf{w} \leftarrow \mathbf{0}$ ▷ the objective is convex: any start works
for $e = 1, \dots, E$ **do**
  shuffle the indices $1, \dots, n$ and cut them into batches B of size n' (the last may be smaller)
  for each batch B **do**
    $\mathbf{g} \leftarrow \frac{2}{|B|} \sum_{i \in B} (\mathbf{w}^\mathsf{T} \tilde{\mathbf{x}}_i - y_i)\, \tilde{\mathbf{x}}_i$ ▷ $O(n'p)$; $B = \{1, \dots, n\}$ gives batch GD
    $\mathbf{g} \leftarrow \mathbf{g} + 2\lambda' D\mathbf{w}$ ▷ $D = \mathrm{diag}(0, 1, \dots, 1)$; $\lambda' = 0$: least squares
    $\mathbf{w} \leftarrow \mathbf{w} - \eta\, \mathbf{g}$
  end for
  optionally decrease η; stop when the validation error no longer improves
end for
return $\mathbf{w}$

Running time — gradient descent for least squares (Algorithm 5)

One full gradient costs two matrix–vector products, $O(np)$; one epoch of mini-batch descent also costs $O(np)$ in total, split into n/n' cheap steps of $O(n'p)$. Batch gradient descent needs about $\kappa \log(1/\epsilon)$ iterations to reach accuracy ϵ, with κ the condition number of $\tilde{X}^\mathsf{T}\tilde{X}$, so its total cost is $O(\kappa\, np \log(1/\epsilon))$ — no p^3 term and no $p \times p$ matrix. Memory: $O(p)$ for the weights plus one batch, $O(n'p)$, so the data can stream from disk. Prediction: $O(d)$ per query. Gradient descent wins when p is large, the data are sparse (each product then costs only the number of non-zero entries) or an approximate answer suffices; for moderate p and ill-conditioned data the direct methods are faster and exact. *Typical sizes*: for $n = 10^8$ and $d = 50$ one epoch is about 10^{10} operations, seconds on a GPU, where each mini-batch step is a pair of dense matrix–vector products that parallelise well.

Other iterative methods. Newton's method (Note 7, Section 6) is exact here: the Hessian $2\tilde{X}^\mathsf{T}\tilde{X}$ is constant, and from *any* $\mathbf{w}$ the Newton step gives $\mathbf{w} - (2\tilde{X}^\mathsf{T}\tilde{X})^{-1}\, 2\tilde{X}^\mathsf{T}(\tilde{X}\mathbf{w} - \mathbf{y}) = (\tilde{X}^\mathsf{T}\tilde{X})^{-1}\tilde{X}^\mathsf{T}\mathbf{y}$, the solution of the normal equations — one Newton step *is* solving them. Between the direct methods and gradient descent sits the *conjugate gradient* method applied to the normal equations (as in the library routines LSQR or LSMR). Like gradient descent it needs only products with $\tilde{X}$ and $\tilde{X}^\mathsf{T}$, $O(np)$ per iteration and $O(n+p)$ extra memory; it reaches the exact solution in at most p iterations in exact arithmetic, and in practice needs about $\sqrt{\kappa}\log(1/\epsilon)$ iterations instead of the $\kappa \log(1/\epsilon)$ of gradient descent [8]. Natural-gradient and adaptive methods matter for non-quadratic objectives and are left to Note 7 and week 5.

3 When least squares is unstable

The solution is unique only if the columns of $\tilde{X}$ are linearly independent, and it is reliable only if they are far from dependent.

- *Collinear features.* If two inputs measure almost the same thing, the data cannot tell how to split the weight between them: $w_1 + w_2$ is well determined, $w_1 - w_2$ is not. Figure 2 shows 300 repetitions of an experiment with $x_2 = x_1 +$ small noise: least-squares estimates of w_1 scatter with a standard deviation of 2.05 around the true value 1, along a line of nearly constant $w_1 + w_2$. Predictions inside the data remain fine; interpretations and predictions away from the data do not.

- *More features than examples.* If $d+1 > n$, $\tilde{X}^{\mathsf{T}}\tilde{X}$ is singular and infinitely many $\mathbf{w}$ fit the training data exactly: adding to $\hat{\mathbf{w}}$ any vector $\mathbf{v}$ with $\tilde{X}\mathbf{v} = \mathbf{0}$ (there are such vectors when the $d+1$ columns live in $\mathbb{R}^n$ with $n < d+1$) changes no prediction. `lstsq` then returns the solution of minimum norm — the implicit preference behind the double-descent curve of Note 4, Section 8.

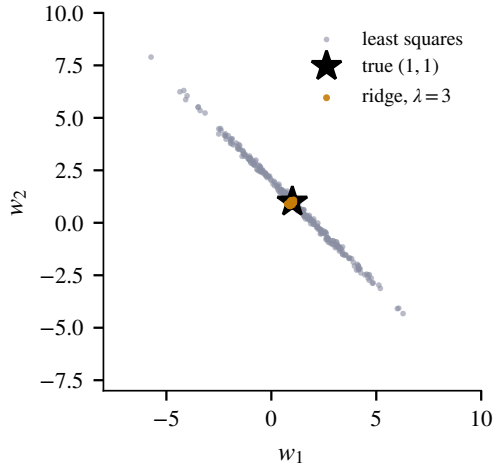


Figure 2. Estimates of (w_1, w_2) from 300 datasets of 30 points with nearly collinear inputs and true weights $(1, 1)$. Least squares (grey) spreads along the poorly determined direction $w_1 - w_2$; ridge regression with $\lambda = 3$ (amber) stays close to the truth: standard deviation of w_1 falls from 2.05 to 0.05, at the price of a small bias (mean 0.95).

Worked example — two thermometers

Two thermometers read almost the same temperature deviation; the two feature columns of four centred readings are

$$\mathbf{f}_1 = (-2, -1, 1, 2), \qquad \mathbf{f}_2 = (-2, -0.9, 0.9, 2).$$

Then $\|\mathbf{f}_1\|^2 = 10$, $\|\mathbf{f}_2\|^2 = 9.62$, $\mathbf{f}_1^{\mathsf{T}}\mathbf{f}_2 = 9.8$, and $X^{\mathsf{T}}X = \left(\begin{smallmatrix} 10 & 9.8 \\ 9.8 & 9.62 \end{smallmatrix}\right)$ has the tiny determinant $10 \cdot 9.62 - 9.8^2 = 0.16$. Consider two target vectors that differ by 0.1 in two entries, as two repetitions of the experiment might. Both inputs and both target vectors are centred (each sums to zero), so by Section 2.3 the bias is $\bar{y} = 0$ and only the 2×2 matrix $X^{\mathsf{T}}X$ is needed:

	targets $\mathbf{y}$	$X^{\mathsf{T}}\mathbf{y}$	least squares $(\hat{w}_1, \hat{w}_2)$
data set A	$(-4.1, -1.8, 2.0, 3.9)$	$(19.8, 19.42)$	$(1.00, 1.00)$
data set B	$(-4.1, -1.9, 2.1, 3.9)$	$(20.0, 19.6)$	$(2.00, 0.00)$

For A, $\hat{\mathbf{w}} = \frac{1}{0.16}\left(\begin{smallmatrix} 9.62 & -9.8 \\ -9.8 & 10 \end{smallmatrix}\right)\left(\begin{smallmatrix} 19.8 \\ 19.42 \end{smallmatrix}\right) = \frac{1}{0.16}\left(\begin{smallmatrix} 0.16 \\ 0.16 \end{smallmatrix}\right)$; for B the same matrix gives $\frac{1}{0.16}(0.32, 0)$. A change of 0.1 in the data moves the weights by 1: the difference $w_1 - w_2$ jumps from 0 to 2, while the sum $w_1 + w_2 = 2$ is the same in both. The singular values of X are 4.43 and 0.090 (the square roots of the eigenvalues 19.61 and 0.0082 of $X^{\mathsf{T}}X$), so $\kappa(X) = 49$ and $\kappa(X^{\mathsf{T}}X) \approx 2400$; the right singular vectors are $(0.714, 0.700)$ (the sum direction) and $(0.700, -0.714)$ (the difference direction). We return to this example with ridge regression in Section 4.1.

4 Ridge regression

Regularisation (Note 4, Section 7) adds to the training loss a penalty $\Omega(\mathbf{w}) \geq 0$ that encodes a preference among parameter values, weighted by $\lambda \geq 0$:

$$J_\lambda(\mathbf{w}) = \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 + \lambda\,\Omega(\mathbf{w}).$$

The choice $\Omega = \sum_j w_j^2$ gives ridge regression (this section), $\Omega = \sum_j |w_j|$ the lasso (Section 5) and a mixture of the two the elastic net (Section 5.4); each Ω is also a prior belief about $\mathbf{w}$ (Section 6).

Ridge regression uses the weight-decay penalty, and was introduced for regression by Hoerl and Kennard [7]:

$$J_\lambda(\mathbf{w}) = \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 + \lambda \sum_{j=1}^{d} w_j^2, \tag{3}$$

leaving the bias unpenalised: shifting all targets by a constant should shift b, not shrink it, and a penalised bias would make the predictions depend on where we put the origin of y [5, Section 3.4.1].

4.1 The closed form, step by step

Write the penalty as a quadratic form, $\sum_{j=1}^{d} w_j^2 = \mathbf{w}^\mathsf{T} D \mathbf{w}$, where $D = \mathrm{diag}(0, 1, \dots, 1)$ is the $(d+1) \times (d+1)$ diagonal matrix that skips the bias entry and keeps the weights. Then

$$J_\lambda(\mathbf{w}) = \mathbf{w}^\mathsf{T}(\tilde{X}^\mathsf{T}\tilde{X} + \lambda D)\mathbf{w} - 2(\tilde{X}^\mathsf{T}\mathbf{y})^\mathsf{T}\mathbf{w} + \mathbf{y}^\mathsf{T}\mathbf{y},$$

and the two rules of Section 2.2 give

$$\nabla J_\lambda = 2\tilde{X}^\mathsf{T}(\tilde{X}\mathbf{w} - \mathbf{y}) + 2\lambda D\mathbf{w} = \mathbf{0} \quad \Longrightarrow \quad \hat{\mathbf{w}}_\lambda = (\tilde{X}^\mathsf{T}\tilde{X} + \lambda D)^{-1}\tilde{X}^\mathsf{T}\mathbf{y}.$$

The penalty simply adds λ to the diagonal of the Gram matrix, except for the bias entry. For $\lambda > 0$ the matrix is positive definite, so the solution is unique even when $d > n$. To see this, take any $\mathbf{v} = (v_0, v_1, \dots, v_d) \neq \mathbf{0}$, with v_0 the bias component; then $\mathbf{v}^\mathsf{T}(\tilde{X}^\mathsf{T}\tilde{X} + \lambda D)\mathbf{v} = \left\|\tilde{X}\mathbf{v}\right\|^2 + \lambda \sum_{j\geq 1} v_j^2$. If some v_j with $j \geq 1$ is non-zero, the second term is positive. Otherwise $\mathbf{v} = (v_0, 0, \dots, 0)$ with $v_0 \neq 0$, so $\tilde{X}\mathbf{v} = v_0\mathbf{1}$ and $\left\|\tilde{X}\mathbf{v}\right\|^2 = n v_0^2 > 0$. Because D has a zero in the bias position, the first equation of $(\tilde{X}^\mathsf{T}\tilde{X} + \lambda D)\mathbf{w} = \tilde{X}^\mathsf{T}\mathbf{y}$ is the same as in least squares, so with centred data (Section 2.3) it again gives $\hat{b} = \bar{y} - \hat{\mathbf{w}}^\mathsf{T}\bar{\mathbf{x}}$, and the weights are

$$\hat{\mathbf{w}}_\lambda = (X_c^\mathsf{T}X_c + \lambda I)^{-1}X_c^\mathsf{T}\mathbf{y}_c. \tag{4}$$

For a single centred input this is

$$\hat{w}_\lambda = \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda} = \frac{S_{xx}}{S_{xx} + \lambda}\,\hat{w}_{\mathrm{LS}},$$

the least-squares slope multiplied by a *shrinkage factor* between 0 and 1.

Worked example — ridge shrinkage in one dimension

For the cart data ($S_{xy} = 22$, $S_{xx} = 10$, $\bar{x} = 2$, $\bar{y} = 5$), $\hat{w}_\lambda = 22/(10 + \lambda)$ and, since the bias is not penalised, $\hat{b}_\lambda = 5 - 2\hat{w}_\lambda$:

λ	0	1	5	10	40
shrinkage factor $10/(10+\lambda)$	1	0.909	0.667	0.5	0.2
slope $\hat{w}_\lambda$ (cm/s)	2.20	2.00	1.47	1.10	0.44
intercept $\hat{b}_\lambda$ (cm)	0.60	1.00	2.07	2.80	4.12

At $\lambda = S_{xx} = 10$ the slope is exactly halved. The slope never reaches zero for finite λ, and every line passes through $(\bar{x}, \bar{y})$ (Figure 1, right). The penalty is measured in the units of $\sum_i x_i^2$: had time been recorded in tenths of a second, S_{xx} would be 100 times larger and $\lambda = 10$ would barely shrink anything.

Worked example — ridge tames the two thermometers

Return to the thermometers of Section 3 with $\lambda = 1$. Now $X^{\mathsf{T}}X+I = \left(\begin{smallmatrix} 11 & 9.8 \\ 9.8 & 10.62 \end{smallmatrix}\right)$ has determinant $116.82 - 96.04 = 20.78$ instead of 0.16, and (4) gives

$$\text{A: } \hat{\mathbf{w}}_{\lambda=1} = \frac{1}{20.78}\left(\begin{smallmatrix} 10.62 & -9.8 \\ -9.8 & 11 \end{smallmatrix}\right)\left(\begin{smallmatrix} 19.8 \\ 19.42 \end{smallmatrix}\right) = (0.961, 0.942), \qquad \text{B: } \hat{\mathbf{w}}_{\lambda=1} = (0.978, 0.943).$$

The two data sets now give almost the same answer: the jump of 1 in each weight has become a change of less than 0.02. The price is a bias: the sums $w_1 + w_2 = 1.90$ and 1.92 are slightly smaller than the least-squares value 2. This is the bias–variance trade-off of Note 4, Section 6, in two lines of arithmetic.

4.2 What ridge does: the SVD picture

Centre the inputs and targets and write the thin singular value decomposition $X = USV^{\mathsf{T}}$ (we drop the subscript c), with $U \in \mathbb{R}^{n\times d}$ and $V \in \mathbb{R}^{d\times d}$ having orthonormal columns $\mathbf{u}_j$ and $\mathbf{v}_j$, $U^{\mathsf{T}}U = V^{\mathsf{T}}V = VV^{\mathsf{T}} = I$, and $S = \operatorname{diag}(s_1, \dots, s_d)$ with $s_1 \geq s_2 \geq \dots \geq 0$ (we assume $n \geq d$). Equivalently, the s_j^2 are the eigenvalues of $X^{\mathsf{T}}X$ and the $\mathbf{v}_j$ its eigenvectors, the principal axes of the cloud of centred inputs: s_j^2/n is the variance of the data along $\mathbf{v}_j$. For the thermometers, $\mathbf{v}_1$ is the sum direction, with variance $19.61/4 = 4.90$, and $\mathbf{v}_2$ the difference direction, with variance 0.002. Each step of the derivation uses only the orthogonality relations:

$$\begin{aligned} X^{\mathsf{T}}X + \lambda I &= VSU^{\mathsf{T}}USV^{\mathsf{T}} + \lambda VV^{\mathsf{T}} = V(S^2 + \lambda I)V^{\mathsf{T}}, \\ (X^{\mathsf{T}}X + \lambda I)^{-1} &= V(S^2 + \lambda I)^{-1}V^{\mathsf{T}} \qquad \text{(invert the diagonal, keep the rotations)}, \\ \hat{\mathbf{w}}_\lambda &= V(S^2 + \lambda I)^{-1}V^{\mathsf{T}}\, VSU^{\mathsf{T}}\mathbf{y} = \sum_{j=1}^{d} \mathbf{v}_j \frac{s_j}{s_j^2 + \lambda}\, \mathbf{u}_j^{\mathsf{T}}\mathbf{y}, \\ \hat{\mathbf{y}}_\lambda &= X\hat{\mathbf{w}}_\lambda = USV^{\mathsf{T}}V(S^2 + \lambda I)^{-1}SU^{\mathsf{T}}\mathbf{y} = \sum_{j=1}^{d} \mathbf{u}_j \frac{s_j^2}{s_j^2 + \lambda}\, \mathbf{u}_j^{\mathsf{T}}\mathbf{y}. \end{aligned}$$

The sums come from one rule: for any diagonal matrix M, $VMU^{\mathsf{T}}\mathbf{y} = \sum_j \mathbf{v}_j\, m_j\, (\mathbf{u}_j^{\mathsf{T}}\mathbf{y})$, because $U^{\mathsf{T}}\mathbf{y}$ lists the components of $\mathbf{y}$ along the $\mathbf{u}_j$, M scales each of them and V reassembles the scaled components along the $\mathbf{v}_j$ (here $m_j = s_j/(s_j^2 + \lambda)$; for $\hat{\mathbf{y}}_\lambda$, V is replaced by U). Least squares ($\lambda = 0$) projects $\mathbf{y}$ onto each direction $\mathbf{u}_j$ and keeps it entirely; ridge shrinks each one by the factor $s_j^2/(s_j^2 + \lambda)$. Directions in which the inputs vary a lot (large s_j) are kept, directions of little variation — where the estimate was noise, as for $w_1 - w_2$ in Figure 2 — are suppressed. For the coefficients, least squares multiplies by $1/s_j$, which is why a tiny singular value made the weights explode for the two thermometers; ridge multiplies by $s_j/(s_j^2 + \lambda) = 1/(s_j + \lambda/s_j)$. The function $s + \lambda/s$ has derivative $1 - \lambda/s^2$, which vanishes at $s = \sqrt{\lambda}$, so its minimum is $2\sqrt{\lambda}$ and the ridge factor never exceeds $1/(2\sqrt{\lambda})$, however small s_j is.

The sum $\mathrm{df}(\lambda) = \sum_j s_j^2/(s_j^2 + \lambda)$ is the *effective number of parameters*, which decreases continuously from $\operatorname{rank}(X)$ (= d for full rank) at $\lambda = 0$ to 0 as $\lambda \to \infty$: the effective capacity of Note 4, Section 5, made exact. For the thermometers, $s_1^2 = 19.61$ and $s_2^2 = 0.0082$; with $\lambda = 1$ the sum direction is kept with factor 0.951 and the difference direction with factor 0.008, so $\mathrm{df}(1) = 0.96$: the two thermometers count as a single parameter, which is all the data can determine. Because the penalty treats all w_j alike, features must be put on comparable scales first (Note 5, Section 5); otherwise λ penalises a coefficient merely for its units.

4.3 Ridge as weight decay

Ridge can also be minimised by gradient descent. With centred X and $\mathbf{y}$ (Section 2.3), so that the bias is not decayed, and the penalty added to the mean loss L, the objective is $\frac{1}{n}\|X\mathbf{w}-\mathbf{y}\|^2+\lambda'\|\mathbf{w}\|^2$ and the update is

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w}-\eta\Big(\tfrac{2}{n}X^{\mathsf{T}}(X\mathbf{w}-\mathbf{y})+2\lambda'\mathbf{w}\Big)\\ &= (1-2\eta\lambda')\,\mathbf{w}-\eta\,\tfrac{2}{n}X^{\mathsf{T}}(X\mathbf{w}-\mathbf{y})\ :\end{aligned}$$

before each ordinary gradient step the weights are multiplied by a factor slightly below one — they *decay*. Mind the normalisation: this objective is (3) divided by n, with $\lambda=n\lambda'$, so the same solution needs a penalty n times smaller here; for the cart ($n=5$), $\lambda=10$ in (3) corresponds to $\lambda'=2$. With mini-batches, Algorithm 5 adds the same term $2\lambda' D\mathbf{w}$ to every batch gradient: because the objective uses the *mean* loss, the batch average and the penalty are on the same scale, and no splitting of the penalty among batches is needed. The decay form is the origin of the name used for neural networks. For plain gradient descent the L_2 penalty and multiplicative decay are the same thing; AdamW applies the decay step directly, decoupled from its adaptive gradient step, and for adaptive methods this is not equivalent to an L_2 penalty (Note 7, Section 6). Stopping plain gradient descent early from $\mathbf{w}=\mathbf{0}$ (Note 7, Section 7) has an effect similar to ridge. Along the eigendirection $\mathbf{v}_j$, the component of $\mathbf{w}$ after k steps of size η on the mean loss is $\big(1-(1-2\eta s_j^2/n)^k\big)$ times its least-squares value: directions with large s_j are fitted within a few steps, those with small s_j have barely moved when training stops — the same pattern as the ridge factor $s_j^2/(s_j^2+\lambda)$.

Algorithm 6 Ridge regression, closed form, for one or many values of λ

Require: $X\in\mathbb{R}^{n\times d}$, $\mathbf{y}\in\mathbb{R}^n$, penalties $\lambda^{(1)},\dots,\lambda^{(M)}>0$
Ensure: $(\hat b_\lambda,\hat{\mathbf{w}}_\lambda)$ for each λ

$\bar{\mathbf{x}}\leftarrow$ column means of X, $\bar y\leftarrow$ mean of $\mathbf{y}$; $X_c\leftarrow X-\mathbf{1}\bar{\mathbf{x}}^{\mathsf{T}}$, $\mathbf{y}_c\leftarrow\mathbf{y}-\bar y\mathbf{1}$
optionally divide each column j of X_c by its standard deviation σ_j ▷ Note 5, Section 5
if only one λ **then**
  solve $(X_c^{\mathsf{T}}X_c+\lambda I)\hat{\mathbf{w}}=X_c^{\mathsf{T}}\mathbf{y}_c$ by Cholesky ▷ as Algorithm 2
else
  $(U,\mathbf{s},V)\leftarrow\textsc{ThinSVD}(X_c)$; $\mathbf{z}\leftarrow U^{\mathsf{T}}\mathbf{y}_c$ ▷ once, $O(nd\,r)$, $r=\min(n,d)$
  for $\ell=1,\dots,M$ **do**
    $\hat{\mathbf{w}}_{\lambda^{(\ell)}}\leftarrow V\big(z_js_j/(s_j^2+\lambda^{(\ell)})\big)_{j=1}^{r}$ ▷ $V\in\mathbb{R}^{d\times r}$; $O(dr)$ per value
  end for
end if
if the columns were standardised: $\hat w_j\leftarrow\hat w_j/\sigma_j$ for each j and λ ▷ back to original units
$\hat b_\lambda\leftarrow\bar y-\hat{\mathbf{w}}_\lambda^{\mathsf{T}}\bar{\mathbf{x}}$ for each λ ▷ the bias is not shrunk
return all $(\hat b_\lambda,\hat{\mathbf{w}}_\lambda)$; choose λ on validation data (Note 4, Section 7)

Running time — ridge regression (Algorithm 6)

One λ: the same as least squares, $O(nd^2+d^3)$ time and $O(d^2)$ extra memory; adding λ to the diagonal costs nothing and improves the conditioning, so the Cholesky factorisation never fails. *Many* λ (for validation): one thin SVD, $O(nd\min(n,d))$, then $O(d\min(n,d))$ per value instead of $O(nd^2+d^3)$ — a whole regularisation path for the price of one fit. *More features than examples* ($d>n$): the identity $(X^{\mathsf{T}}X+\lambda I)^{-1}X^{\mathsf{T}}=X^{\mathsf{T}}(XX^{\mathsf{T}}+\lambda I)^{-1}$ gives $\hat{\mathbf{w}}=X^{\mathsf{T}}(XX^{\mathsf{T}}+\lambda I)^{-1}\mathbf{y}$, which needs an $n\times n$ instead of a $d\times d$ system, $O(n^2d+n^3)$; this *dual* form is the starting point of kernel ridge regression in Note 10, Section 6. *Prediction*: $O(d)$ per query. By gradient descent (Section 4.3): $O(nd)$ per iteration, and fewer iterations than unregularised least squares because λ lifts the smallest eigenvalue of the Hessian.

5 The lasso and sparse models

Replacing the squared penalty with absolute values gives the *lasso* [9],

$$J_\lambda(\mathbf{w}) = \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 + \lambda \sum_{j=1}^{d} |w_j|,$$

again with an unpenalised bias, which we remove by centring. The L_1 penalty has a kink at zero, and for large enough λ many coefficients become exactly zero: the lasso selects features. There is no closed form in general, but the one-dimensional problem has one, and it is the building block of a simple and fast algorithm.

5.1 One coefficient: soft thresholding

Consider a single centred feature column $\mathbf{f} = (x_1, \dots, x_n)$ and target (or residual) vector $\mathbf{r}$, and minimise

$$q(w) = \sum_{i=1}^{n} (r_i - w x_i)^2 + \lambda|w| = c\, w^2 - 2\rho\, w + \textstyle\sum_i r_i^2 + \lambda|w|, \qquad c = \textstyle\sum_i x_i^2, \quad \rho = \textstyle\sum_i x_i r_i.$$

Without the penalty the minimiser would be ρ/c. The function q is convex (a convex quadratic plus $\lambda|w|$), so a point where it stops decreasing and starts increasing is its global minimum, and there is only one such point (for $c > 0$). The function $|w|$ is differentiable everywhere except at 0, so we treat three cases; at most one of the first two can hold, since $\rho > \lambda/2$ and $\rho < -\lambda/2$ exclude each other.

- $w > 0$: here $|w| = w$ and $q'(w) = 2cw - 2\rho + \lambda = 0$ gives $w = (\rho - \lambda/2)/c$. This is consistent (positive) only if $\rho > \lambda/2$.
- $w < 0$: here $|w| = -w$ and $q'(w) = 2cw - 2\rho - \lambda = 0$ gives $w = (\rho + \lambda/2)/c$, consistent only if $\rho < -\lambda/2$.
- Otherwise, $|\rho| \le \lambda/2$: for $w > 0$, $q'(w) = 2cw + (\lambda - 2\rho) > 0$ because $\lambda/2 \ge \rho$; for $w < 0$, $q'(w) = 2cw - (\lambda + 2\rho) < 0$ because $\rho \ge -\lambda/2$. So q decreases up to 0 and increases after it: the minimum sits at the kink, $w = 0$. (For readers who know subgradients: the slope of $\lambda|w|$ at 0 can be anything in $[-\lambda, \lambda]$, and $0 \in -2\rho + [-\lambda, \lambda]$ exactly when $|\rho| \le \lambda/2$.)

The three cases combine into the *soft-thresholding* formula

$$\hat{w} = \frac{\mathrm{S}(\rho, \lambda/2)}{c}, \qquad \mathrm{S}(z, t) = \mathrm{sign}(z)\max(|z| - t, 0) = \begin{cases} z - t & z > t, \\ 0 & |z| \le t, \\ z + t & z < -t. \end{cases} \tag{5}$$

Compare ridge in the same setting: $\hat{w} = \rho/(c + \lambda)$, a *proportional* shrinkage that is never exactly zero unless $\rho = 0$. The lasso instead subtracts a *constant* amount $\lambda/2$ from $|\rho|$ and stops at zero. When the design is orthonormal ($X^\mathsf{T}X = I$) the coordinates decouple. Indeed, with $\mathbf{a} = X^\mathsf{T}\mathbf{y}$ (the least-squares solution, since $X^\mathsf{T}X = I$),

$$\|X\mathbf{w} - \mathbf{y}\|^2 = \mathbf{w}^\mathsf{T}\mathbf{w} - 2\mathbf{a}^\mathsf{T}\mathbf{w} + \mathbf{y}^\mathsf{T}\mathbf{y} = \sum_j (w_j - a_j)^2 + \text{const},$$

so each w_j appears in its own term, which is the one-dimensional problem above with $c = 1$ and $\rho = a_j$. The exact ridge and lasso solutions are therefore $a_j/(1 + \lambda)$ and $\mathrm{S}(a_j, \lambda/2)$, with $a_j = \mathbf{f}_j^\mathsf{T}\mathbf{y}$ the least-squares coefficients (Figure 3, left).

Worked example — soft thresholding with numbers

(a) The cart slope. With $\rho = S_{xy} = 22$ and $c = S_{xx} = 10$:

λ	4	20	44	50
lasso $S(22, \lambda/2)/10$	$20/10 = 2.0$	$12/10 = 1.2$	0	0
ridge $22/(10+\lambda)$	1.57	0.73	0.41	0.37

The lasso slope vanishes as soon as $\lambda/2 \geq |\rho|$, i.e. $\lambda \geq 44$; the ridge slope only approaches zero. *(b) An orthonormal design.* Suppose $X^{\mathsf{T}}X = I$ and the least-squares coefficients are $\mathbf{a} = (3.0, -1.2, 0.4, -0.1)$. With $\lambda = 1$ (threshold $\lambda/2 = 0.5$), the lasso gives $(2.5, -0.7, 0, 0)$: the two small coefficients are removed, the large ones are moved towards zero by 0.5. Ridge gives $\mathbf{a}/2 = (1.5, -0.6, 0.2, -0.05)$: everything halved, nothing removed. These are the dots in Figure 3, left.

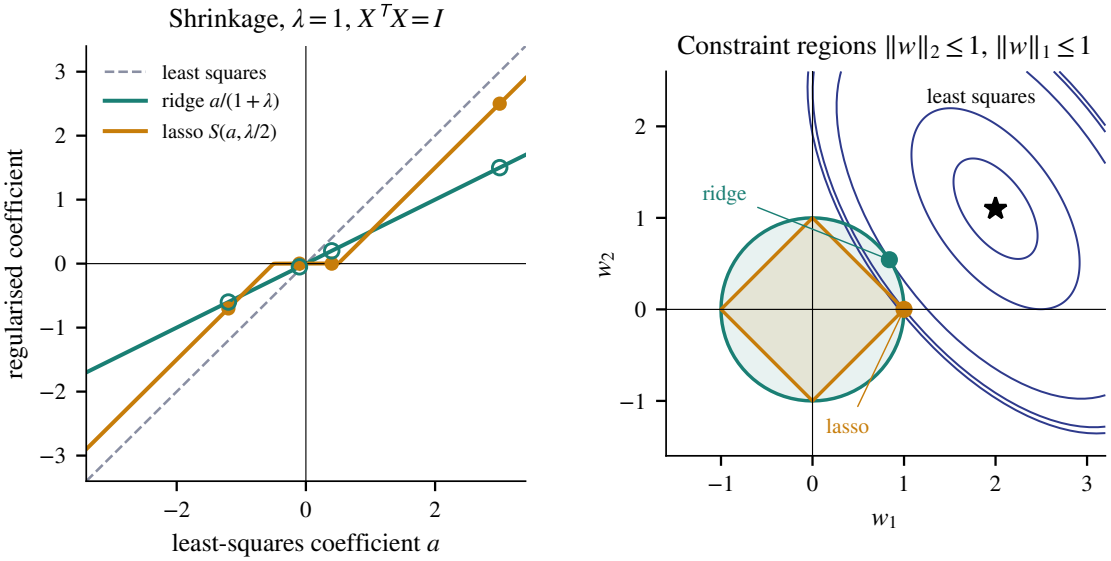


Figure 3. Left: for an orthonormal design, ridge multiplies each least-squares coefficient by $1/(1+\lambda)$, the lasso moves it towards zero by $\lambda/2$ and sets it to zero inside $[-\lambda/2, \lambda/2]$; the markers (filled: lasso, open: ridge) are part (b) of the worked example in Section 5.1. Right: the constrained form. Contours of the squared error (indigo) grow from the least-squares point until they touch the constraint region; the L_2 disc is touched at a generic point $(0.84, 0.54)$, the L_1 diamond at its corner $(1, 0)$, where $w_2 = 0$ exactly.

5.2 Why exact zeros: the geometry

With the norms of Note 1, Section 6, $\|\mathbf{w}\|_1 = \sum_j |w_j|$ and $\|\mathbf{w}\|_2^2 = \sum_j w_j^2$, minimising $\|X\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_1$ is equivalent to minimising $\|X\mathbf{w} - \mathbf{y}\|^2$ subject to $\|\mathbf{w}\|_1 \leq t$ for a suitable budget t (each λ corresponds to some t, by Lagrange multipliers; the same holds for ridge with $\|\mathbf{w}\|_2^2 \leq t$). The solution is where the growing elliptical contours of the squared error first touch the constraint region (Figure 3, right). The L_2 ball is round, and the contact point is generically off the axes: both weights non-zero. The L_1 ball is a diamond (the L_1 unit ball of Note 5, Section 5; in d dimensions a *cross-polytope*) whose corners lie on the axes and stick out. In two dimensions a typical ellipse often touches it first at a corner, where one coordinate is exactly zero (on the edges between corners both coordinates are non-zero). In d dimensions the contact is often at a corner or on a low-dimensional face — in three dimensions an edge of the octahedron, where one coordinate is zero — so several coordinates vanish. The L_1 ball has ever more corners and low-dimensional faces as d grows, which is the geometric intuition for

Algorithm 7 Lasso (and elastic net) by cyclic coordinate descent

Require: $X \in \mathbb{R}^{n\times d}$, $\mathbf{y}$, penalty $\lambda > 0$, optional $\lambda_2 \geq 0$ (default 0), tolerance ϵ
Ensure: $(\hat{b}, \hat{\mathbf{w}})$ minimising $\|\mathbf{y} - b\mathbf{1} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_1 + \lambda_2\|\mathbf{w}\|_2^2$ (penalties on the standardised weights if the columns are standardised; $\lambda_2 = 0$ is the lasso)

centre (and usually standardise, by σ_j) X and $\mathbf{y}$ as in Algorithm 6, giving X_c, $\mathbf{y}_c$
$\mathbf{w} \leftarrow \mathbf{0}$ (or the solution for a nearby λ: *warm start*); $\mathbf{r} \leftarrow \mathbf{y}_c - X_c\mathbf{w}$
$c_j \leftarrow \|\mathbf{f}_j\|^2$ for $j = 1, \dots, d$ ▷ columns $\mathbf{f}_j$ of X_c; $O(nd)$, once
repeat
  for $j = 1, \dots, d$ **do**
    $\rho_j \leftarrow \mathbf{f}_j^{\mathsf{T}}\mathbf{r} + c_j w_j$ ▷ correlation with the partial residual, $O(n)$
    $w^{\text{new}} \leftarrow \mathrm{S}(\rho_j, \lambda/2)/(c_j + \lambda_2)$ ▷ soft threshold (5)
    $\mathbf{r} \leftarrow \mathbf{r} - \mathbf{f}_j\,(w^{\text{new}} - w_j)$; $w_j \leftarrow w^{\text{new}}$ ▷ $O(n)$; skip if unchanged
  end for
until no w_j changed by more than ϵ during the sweep
if the columns were standardised: $w_j \leftarrow w_j/\sigma_j$ for each j ▷ back to original units
$\hat{b} \leftarrow \bar{y} - \mathbf{w}^{\mathsf{T}}\bar{\mathbf{x}}$
return $(\hat{b}, \mathbf{w})$

why lasso solutions are typically sparse; how sparse they are in a given problem depends on λ, on the correlations between the features and on the signal.

5.3 Coordinate descent

With several features, minimise over one coordinate at a time, keeping the others fixed. For coordinate j, the other features contribute a fixed prediction, and what w_j must explain is the *partial residual* $\mathbf{r}^{(j)} = \mathbf{y}_c - \sum_{k\neq j} w_k\mathbf{f}_k$, where $\mathbf{f}_k$ denotes column k of X_c (Section 2.3). This is exactly the one-dimensional problem of Section 5.1. With the full residual $\mathbf{r} = \mathbf{y}_c - X_c\mathbf{w}$ we have $\mathbf{r}^{(j)} = \mathbf{r} + w_j\mathbf{f}_j$, hence

$$w_j \leftarrow \frac{\mathrm{S}(\rho_j, \lambda/2)}{c_j}, \qquad \rho_j = \mathbf{f}_j^{\mathsf{T}}\mathbf{r}^{(j)} = \mathbf{f}_j^{\mathsf{T}}\mathbf{r} + c_j w_j, \qquad c_j = \|\mathbf{f}_j\|^2 .$$

Keeping $\mathbf{r}$ up to date after every change, $\mathbf{r} \leftarrow \mathbf{r} - \mathbf{f}_j(w_j^{\text{new}} - w_j^{\text{old}})$, makes each coordinate update cost $O(n)$. Each update can only decrease the convex objective, and cycling through the coordinates converges to the lasso solution [3]. At the solution, every zero coefficient satisfies $|\mathbf{f}_j^{\mathsf{T}}\mathbf{r}| \leq \lambda/2$ and every non-zero one $\mathbf{f}_j^{\mathsf{T}}\mathbf{r} = \frac{\lambda}{2}\operatorname{sign}(w_j)$: the correlation of each active feature with the residual is exactly at the threshold.

Worked example — coordinate descent by hand

Fit the lasso with $\lambda = 4$ (threshold $\lambda/2 = 2$) to the detector data of Section 2.3. The features are already centred, $c_1 = 10$, $c_2 = 4$, and $\mathbf{y}_c = \mathbf{y} - 4 = (-3, -2, 0, 1, 4)$. Start from $\mathbf{w} = (0, 0)$, so $\mathbf{r} = \mathbf{y}_c$.

1. $j = 1$: $\rho_1 = \mathbf{f}_1^{\mathsf{T}}\mathbf{r} + 10 \cdot 0 = 6 + 2 + 0 + 1 + 8 = 17$, so $w_1 = \mathrm{S}(17, 2)/10 = 15/10 = 1.5$. Update $\mathbf{r} \leftarrow \mathbf{r} - 1.5\,\mathbf{f}_1 = (0, -0.5, 0, -0.5, 1)$.
2. $j = 2$: $\rho_2 = \mathbf{f}_2^{\mathsf{T}}\mathbf{r} + 4 \cdot 0 = 0 + 0.5 + 0 + 0 + 1 = 1.5$. Since $|1.5| \leq 2$, $w_2 = 0$: the second feature is switched off and $\mathbf{r}$ does not change.
3. Second sweep: $\rho_1 = \mathbf{f}_1^{\mathsf{T}}\mathbf{r} + 10 \cdot 1.5 = 2 + 15 = 17$ again, so $w_1 = 1.5$; $\rho_2 = 1.5$ again, so $w_2 = 0$. Nothing changed: converged.

The lasso solution is $\hat{\mathbf{w}} = (1.5, 0)$ with $\hat{b} = 4$, compared with least squares $(1.667, 0.333)$ and ridge with the same $\lambda = 4$, $(X^{\mathsf{T}}X + 4I)^{-1}X^{\mathsf{T}}\mathbf{y}_c = \left(\begin{smallmatrix}14 & 1\\ 1 & 8\end{smallmatrix}\right)^{-1}\left(\begin{smallmatrix}17\\ 3\end{smallmatrix}\right) = (1.198, 0.225)$ (here $X^{\mathsf{T}}\mathbf{y}_c = X^{\mathsf{T}}\mathbf{y}$ because the columns are centred). The optimality conditions hold: for the active feature $\mathbf{f}_1^{\mathsf{T}}\mathbf{r} = 2 = \lambda/2$, for the inactive one $|\mathbf{f}_2^{\mathsf{T}}\mathbf{r}| = 1.5 \le 2$.

Running time — lasso by coordinate descent (Algorithm 7)

One sweep over all d coordinates costs $O(nd)$: each coordinate needs one dot product with the residual and one residual update, both $O(n)$. The number of sweeps T is typically tens to a few hundred, so training costs $O(Tnd)$, with no d^3 term. Memory: the data, $O(nd)$, plus $O(n)$ for the residual and $O(d)$ for the weights. Large savings come from *warm starts* along a decreasing sequence of λ (each solution is a good start for the next) and from sweeping mostly over the currently non-zero coordinates; computing a whole path then costs little more than one fit [3]. With a sparse X each sweep costs only $O(\mathrm{nnz}(X))$, the number of non-zero entries. The elastic-net term λ_2 only changes a denominator and leaves all costs unchanged. The updates are sequential over j, so GPUs help little; the method is fast because each step is cheap. *Prediction*: $O(k)$ per query, where k is the number of non-zero coefficients — a sparse model is also a cheap model.

5.4 Elastic net and regularisation paths

The *elastic net* adds both penalties, $\lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$, keeping the sparsity of the lasso while behaving better with groups of correlated features [10]: the lasso tends to pick one member of a correlated group more or less arbitrarily (the two thermometers again), the L_2 part spreads the weight over the group. Coordinate descent handles it with a one-line change, $w_j \leftarrow \mathrm{S}(\rho_j, \lambda_1/2)/(c_j + \lambda_2)$: the extra term $\lambda_2 w_j^2$ adds λ_2 to the coefficient c_j of w_j^2 in the one-dimensional problem (Algorithm 7 with $\lambda = \lambda_1$).

Worked example — lasso and elastic net on the two thermometers

Take data set A of the thermometer example (Section 3), whose inputs and targets are already centred, with $\lambda_1 = 2$ (threshold 1). The lasso by coordinate descent from $\mathbf{w} = (0, 0)$: $\rho_1 = \mathbf{f}_1^{\mathsf{T}}\mathbf{y} = 19.8$, so $w_1 = \mathrm{S}(19.8, 1)/10 = 1.88$. The new residual is $\mathbf{r} = \mathbf{y} - 1.88\,\mathbf{f}_1 = (-0.34, 0.08, 0.12, 0.14)$ and $\rho_2 = \mathbf{f}_2^{\mathsf{T}}\mathbf{r} = 0.996 \le 1$, so $w_2 = 0$. A second sweep changes nothing: the lasso answer is $(1.88, 0)$. All the weight goes to the first thermometer, although the data cannot tell the two apart (data set B gives $(1.90, 0)$). Adding $\lambda_2 = 1$ turns the second thermometer back on: coordinate descent now needs many sweeps, because the two coordinates are strongly coupled, and converges to the elastic-net solution $(0.921, 0.885)$ — two similar weights, like ridge, whose sum 1.81 is shrunk somewhat below the well-determined total $w_1 + w_2 \approx 2$. Neither weight is exactly zero, but in a larger problem the irrelevant features would still be removed by the L_1 part.

Figure 4 follows the coefficients of eight correlated features, of which three matter, as λ grows. Ridge shrinks all of them smoothly and none reaches zero. The lasso removes the irrelevant ones first: eight non-zero coefficients at $\lambda = 1$, five at $\lambda = 10$, four at 25 and only the three true ones at $\lambda = 100$. Which λ to use is decided on validation data (Note 4, Section 7).

6 The Bayesian reading

Model the targets as $y_i = \mathbf{w}^{\mathsf{T}}\tilde{\mathbf{x}}_i + \varepsilon_i$ with independent Gaussian noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, and put a prior $p(\mathbf{w})$ on the weights. The derivation of the *maximum a posteriori* (MAP) estimate takes four steps.

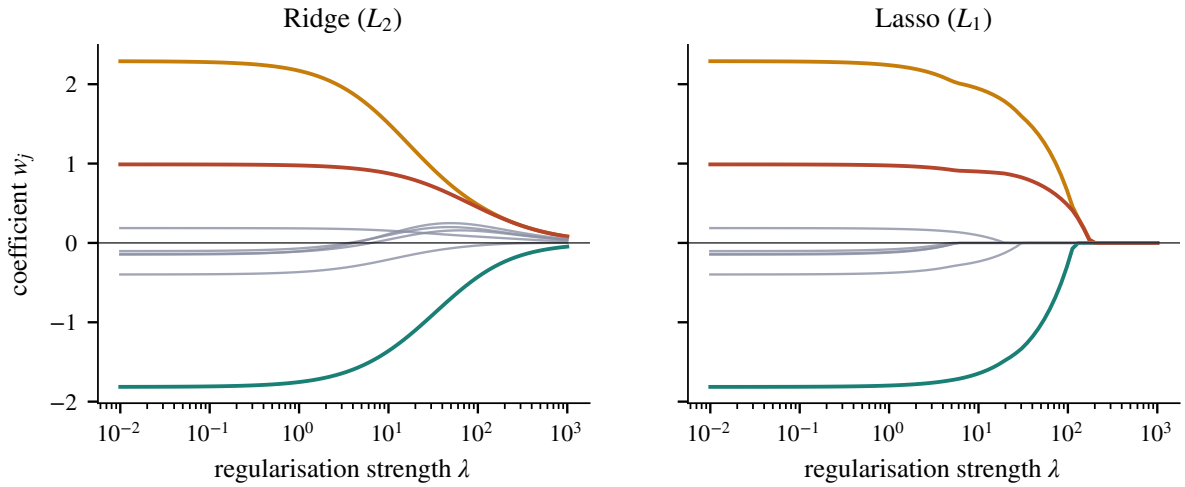


Figure 4. Regularisation paths for $n = 60$ examples with eight standardised, correlated features; coloured lines are the three features with non-zero true weights $(2, -1.5, 1)$, grey lines the five irrelevant ones.

1. Bayes' rule (Note 2, Section 1): $p(\mathbf{w} \mid \text{data}) = p(\text{data} \mid \mathbf{w})\, p(\mathbf{w})/p(\text{data})$, and the denominator does not depend on $\mathbf{w}$.
2. Likelihood (as in Note 2, Section 3): by independence, $p(\text{data} \mid \mathbf{w}) = \prod_i \frac{1}{\sqrt{2\pi}\sigma} \exp\!\big(-(y_i - \mathbf{w}^{\mathsf{T}}\tilde{\mathbf{x}}_i)^2/(2\sigma^2)\big)$.
3. Take $-\log$, which turns the product into a sum and maximisation into minimisation:

$$-\log p(\mathbf{w} \mid \text{data}) = \frac{1}{2\sigma^2}\left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 - \log p(\mathbf{w}) + \text{const.}$$

4. Multiply by $2\sigma^2$, which does not move the minimiser:

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\min_{\mathbf{w}} \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 - 2\sigma^2 \log p(\mathbf{w}).$$

A flat prior gives back least squares. A Gaussian prior $w_j \sim \mathcal{N}(0, \tau^2)$ on the weights (and a flat one on the bias) gives $-\log p(\mathbf{w}) = \sum_{j=1}^{d} w_j^2/(2\tau^2) + \text{const}$, hence

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\min_{\mathbf{w}} \left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2 + \frac{\sigma^2}{\tau^2}\sum_{j=1}^{d} w_j^2 \;: \qquad \text{ridge with } \lambda = \sigma^2/\tau^2.$$

Strong regularisation means a narrow prior relative to the noise. A Laplace prior $p(w_j) = \frac{1}{2\beta} e^{-|w_j|/\beta}$ gives $-2\sigma^2 \log p(\mathbf{w}) = (2\sigma^2/\beta)\sum_{j=1}^{d} |w_j| + \text{const}$: the lasso with $\lambda = 2\sigma^2/\beta$. Its sharp peak at zero is what produces exact zeros. A regulariser is a prior belief about the parameters, made explicit.

> **Worked example — MAP estimate of the cart velocity**
>
> For the cart, suppose the position measurements have noise variance $\sigma^2 = 0.5\,\text{cm}^2$, and that before the experiment we believed the velocity to be small: $w \sim \mathcal{N}(0, \tau^2)$ with $\tau^2 = 0.25\,(\text{cm/s})^2$. Then $\lambda = \sigma^2/\tau^2 = 2$ and the MAP slope is the ridge slope $22/(10 + 2) = 1.833\,\text{cm/s}$, with $\hat{b} = 5 - 2 \times 1.833 = 1.333\,\text{cm}$. To check this directly, write the log-posterior of the slope up to a constant. Because the bias is not penalised, for every w its best value is $b = \bar{y} - w\bar{x}$ (Section 2.1);

substituting it turns each residual $y_i - b - wx_i$ into $(y_i - \bar{y}) - w(x_i - \bar{x})$, so the centred data enter:

$$\ell(w) = -\frac{1}{2\sigma^2}\sum_i \big((y_i - \bar{y}) - w(x_i - \bar{x})\big)^2 - \frac{w^2}{2\tau^2} = -(50 - 44w + 10w^2) - 2w^2,$$

using $\sum(y_i - \bar{y})^2 = 50$, $\sum(x_i - \bar{x})(y_i - \bar{y}) = 22$, $\sum(x_i - \bar{x})^2 = 10$ and $2\sigma^2 = 1$. Setting $\ell'(w) = 44 - 20w - 4w = 0$ gives $w = 44/24 = 1.833$. Numerically, $\ell(1.7) = -9.88$, $\ell(1.833) = -9.667$, $\ell(2.0) = -10.00$ and, at the least-squares slope, $\ell(2.2) = -11.28$: the prior pulls the estimate towards zero. Since ℓ is quadratic, the posterior is Gaussian: a log-density $-aw^2 + kw + \text{const}$ is that of a Gaussian with variance $1/(2a)$ (compare the exponent $-(w-\mu)^2/(2v)$, whose w^2 coefficient is $1/(2v)$). Here $a = 10 + 2 = 12$, so the variance is $1/24 = 0.0417$; in general $a = (S_{xx} + \lambda)/(2\sigma^2)$, giving $\sigma^2/(S_{xx} + \lambda)$. This is a standard deviation of 0.20 cm/s — an error bar that the point estimate alone does not give. With a Laplace prior of scale $\beta = 0.25$ instead, $\lambda = 2\sigma^2/\beta = 4$ and the MAP slope is the lasso value $\mathrm{S}(22, 2)/10 = 2.0$ of the soft-thresholding example in Section 5.1.

Modern connection — linear probes read out what a network knows

Ridge regression is a standard instrument for asking what information a trained network's activations contain: record the activation vector $\mathbf{x}$ of some layer for many inputs, regress a quantity of interest on it, and evaluate on held-out inputs. Applied to the activations of large language models reading names of places, historical figures, works of art and news headlines, such *linear probes* predict latitude, longitude and dates well, suggesting that these quantities are represented along roughly linear directions [4]. The activation vectors have thousands of components, comparable to the number of labelled examples in many probing studies, so the regularisation of Section 4 is not optional. A good probe fit shows that information is *present*, not that the network *uses* it.

In physics — discovering equations by sparse regression

To find the law $\dot{\mathbf{z}} = F(\mathbf{z})$ behind a measured trajectory, build a large library of candidate terms — $1, z_1, z_2, z_1^2, z_1 z_2, \sin z_1, \ldots$ — evaluated along the data, and regress the measured derivatives on it with a sparsity-promoting method. Most coefficients vanish, and the few that remain form a readable equation. With this approach ("sparse identification of nonlinear dynamics") the Lorenz equations were recovered from simulated trajectories [2]; the original work used sequentially thresholded least squares, and the lasso of Section 5 is a common alternative. Linear regression on non-linear features, with a sparsity prior playing the role of Occam's razor, becomes a tool for model discovery.

7 Algorithms at a glance

Table 1 collects the algorithms of this note: n examples, d features, mini-batch size n', T iterations or sweeps, k non-zero lasso coefficients, κ the condition number of $X^{\mathsf{T}}X$; the boxes above use $p = d + 1$, which is $\approx d$ in these leading terms.

Prediction costs $O(d)$ per query for every method ($O(k)$ for a sparse lasso model). Direct methods are exact and predictable; iterative ones scale, but their iteration counts grow with κ, which standardising reduces.

Table 1. Costs of the regression algorithms of this note (leading terms; $p = d + 1 \approx d$).

Algorithm	Training time	Memory	Use when
Simple regression (Alg. 1)	$O(n)$	$O(1)$	one input; hand calculations
Normal equations (Alg. 2)	$O(nd^2 + d^3)$	$O(d^2)$, streaming	d up to a few thousand, well-conditioned features
QR (Alg. 3)	$O(nd^2)$	$O(nd)$	moderate d, ill-conditioned but full-rank X
SVD, `lstsq` (Alg. 4)	$O(nd \min(n, d))$, larger constant	$O(nd)$	default for moderate d; rank-deficient X, $d \geq n$
Gradient descent, mini-batch SGD (Alg. 5)	$O(nd)$ per epoch, T epochs	$O(n'd)$	huge n or d, sparse data, streaming
Conjugate gradient (LSQR)	$O(nd)$ per iteration, at most $d + 1$ iterations	$O(n + d)$ extra	large sparse X, faster than GD when ill-conditioned
Ridge (Alg. 6)	$O(nd^2 + d^3)$; $O(d \min(n, d))$ per extra λ; dual $O(n^2 d + n^3)$	$O(d^2)$ or $O(n^2)$	collinear features, $d \geq n$, many λ to compare
Lasso / elastic net, coordinate descent (Alg. 7)	$O(nd)$ per sweep, T sweeps	$O(nd)$	feature selection, interpretable sparse models

Summary Linear regression predicts $\mathbf{w}^\mathsf{T}\mathbf{x} + b$; a column of ones absorbs the bias, centring separates it again ($\hat{b} = \bar{y} - \hat{\mathbf{w}}^\mathsf{T}\bar{\mathbf{x}}$). Least squares sets the gradient $2\tilde{X}^\mathsf{T}(\tilde{X}\mathbf{w} - \mathbf{y})$ to zero: the normal equations, an orthogonal projection, the maximum-likelihood fit under Gaussian noise (slope S_{xy}/S_{xx} for one input). Solve by Cholesky, QR/SVD or gradient descent. Collinear features make it unstable, $d \geq n$ undetermined. Ridge ($\lambda \|\mathbf{w}\|^2$) shrinks each SVD direction by $s_j^2/(s_j^2 + \lambda)$; the lasso ($\lambda \|\mathbf{w}\|_1$) soft-thresholds and selects features, by coordinate descent. Both are MAP estimates, under Gaussian and Laplace priors.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. Linear regression is "linear" in what? Which of these can be fitted by linear least squares, possibly after transforming variables: $y = a + bx + cx^2$; $y = a \sin x + b \cos x$; $y = ae^{bx}$ ($y > 0$); $y = a/(1 + e^{-b(x-c)})$, with a, b, c all unknown?

A2. Explain the bias trick in words. Why does the bias get its own treatment in ridge regression and the lasso, and what would go wrong with the predictions if all targets were shifted by +100 and the bias were penalised?

A3. Explain, without formulas, why the least-squares residual vector is orthogonal to every column of the design matrix, and what "the fitted values are a projection of the targets" means.

A4. Two thermometers that read almost the same temperature are both used as features. Explain why the individual least-squares weights are unreliable even though the predictions are good. Which combination of the two weights is well determined, and which is not?

A5. What happens to least squares when there are more parameters than examples ($d + 1 > n$)? Is the solution unique, what does `np.linalg.lstsq` return, and how does adding a ridge penalty change the situation?

A6. Why should one not compute $(\tilde{X}^{\mathsf{T}}\tilde{X})^{-1}$ explicitly? What advantage does a QR or SVD solver have over the normal equations, in terms of the condition number?

A7. Compare ridge and lasso: which one produces coefficients that are exactly zero, and why? Give one explanation using the one-dimensional formulas and one using the shape of the constraint regions.

A8. A feature is a length. Explain what happens to its ridge coefficient, and to how strongly it is penalised, if the length is recorded in millimetres instead of metres, with the same λ. What practical rule follows?

A9. Describe what the ridge and lasso estimates become as $\lambda \to 0$ and as $\lambda \to \infty$. Relate the two limits to underfitting, overfitting, bias and variance (Note 4).

A10. In the Bayesian reading, which prior gives ridge and which gives the lasso? If the measurement noise variance σ^2 doubles while the prior stays the same, how does the equivalent λ change, and why does that make sense?

A11. The period T of a pendulum is measured for several lengths ℓ. Which linear regression, on which transformed variables, estimates the exponent α in $T \propto \ell^{\alpha}$? Least squares on the transformed variables assumes Gaussian noise of constant size on $\log T$: what does that mean for the errors of the measured periods T themselves?

A12. Which algorithm of Table 1 would you choose (a) for $n = 10^8$ examples and $d = 50$ features stored on disk, when you need the exact least-squares answer after a single pass over the disk, (b) for the same data when an approximate answer is enough and you want to update the model as new examples arrive, (c) for a ridge or lasso fit with $n = 500$ and $d = 10^5$? Why?

A13. Gradient descent on a least-squares problem diverges. What is the likely cause, what sets the largest allowed learning rate, and how does standardising the features help?

A14. The lasso is applied to a data set containing three almost identical features. What will it typically do with them, and how does the elastic net change this behaviour?

A15. A linear probe trained on the activations of a neural network predicts the temperature of a simulated system with $R^2 = 0.95$ on held-out data. What does this show, and what does it not show?

A16. Why is the L_2 penalty also called "weight decay"? Explain qualitatively, in terms of which directions of the weight space are fitted quickly and which slowly, why stopping gradient descent early, starting from $\mathbf{w} = \mathbf{0}$, has an effect similar to a ridge penalty.

A17. Why does one sweep of lasso coordinate descent cost only $O(nd)$, with no d^3 term? Why do warm starts along a decreasing sequence of λ values make a whole regularisation path cheap?

A18. A model reaches $R^2 = 0.99$ on its training data. Why is this not enough to conclude that it will predict well on new data? What would you check, and could the training R^2 become negative for least squares with a bias?

B. Short calculations

B1. Fit a line to the points $(x, y) = (1, 2), (2, 3), (3, 5), (4, 6)$: compute $\bar{x}$, $\bar{y}$, S_{xy}, S_{xx}, the slope and the intercept. Compute the four residuals, check that they sum to zero, and compute R^2.

B2. For $J(b, w) = \sum_i (y_i - b - w x_i)^2$, write $\partial J/\partial b$ and $\partial J/\partial w$. Evaluate both at $(b, w) = (0, 1)$ for the data of B1, take one gradient-descent step with $\eta = 0.01$, and check that J decreases (here use the sum J, not the mean loss L of Section 2.5).

B3. For centred data with $\sum_i x_i^2 = 8$ and $\sum_i x_i y_i = 12$, compute the ridge slope for $\lambda = 0, 4, 12$ and the corresponding shrinkage factors. For which λ is the slope half of the least-squares slope?

B4. For an orthonormal design ($X^\mathsf{T} X = I$) the least-squares coefficients are $\mathbf{a} = (2.4, -0.6, 1.1, -3.0)$. Compute the lasso and the ridge coefficients for $\lambda = 2$. How many features does each method keep?

B5. Centred features give $X^\mathsf{T} X = \left(\begin{smallmatrix} 4 & 2 \\ 2 & 2 \end{smallmatrix}\right)$ and $X^\mathsf{T} \mathbf{y} = (6, 4)^\mathsf{T}$. Solve the normal equations, then the ridge equations with $\lambda = 2$. If the uncentred feature means were $\bar{\mathbf{x}} = (1, 2)$ and $\bar{y} = 3$, what are the two intercepts?

B6. The noise variance is $\sigma^2 = 0.2$ and the prior on a single centred slope is Gaussian with mean 0 and variance $\tau^2 = 0.05$; the data give $\sum x_i^2 = 16$ and $\sum x_i y_i = 10$. Compute the equivalent λ, the least-squares slope and the MAP slope. Repeat with a Laplace prior of scale $\beta = 0.2$ instead.

B7. The singular values of a centred design matrix are $(10, 3, 1, 0.1)$. Compute the ridge shrinkage factor of each direction and the effective number of parameters for $\lambda = 1$. (Remember that the factors use s_j^2.)

C. Derivations

C1. Expand $\left\|\tilde{X}\mathbf{w} - \mathbf{y}\right\|^2$, derive its gradient and the normal equations, and show that with a column of ones the residuals sum to zero. Show that for one input the slope is $\widehat{\mathrm{Cov}}(x, y)/\widehat{\mathrm{Var}}(x)$.

C2. Derive the ridge solution, including the matrix D. Show that $\tilde{X}^\mathsf{T}\tilde{X} + \lambda D$ is positive definite for every $\lambda > 0$, even when $d > n$. (Consider $\mathbf{v}^\mathsf{T}(\tilde{X}^\mathsf{T}\tilde{X} + \lambda D)\mathbf{v}$ separately for $\mathbf{v}$ with and without non-zero weight components.)

C3. Using the SVD, show that the ridge fitted values are $\sum_j \mathbf{u}_j s_j^2/(s_j^2 + \lambda)\, \mathbf{u}_j^\mathsf{T}\mathbf{y}$ for centred data, and that the coefficients are $\sum_j \mathbf{v}_j s_j/(s_j^2 + \lambda)\, \mathbf{u}_j^\mathsf{T}\mathbf{y}$.

C4. Show that the minimiser of $(w - a)^2 + \lambda|w|$ is $\mathrm{sign}(a)\max(|a| - \lambda/2, 0)$, treating the cases $w > 0$, $w < 0$ and $w = 0$, and explain why the L_2 penalty never produces exact zeros.

C5. Prove the identity $(X^\mathsf{T}X + \lambda I_d)^{-1}X^\mathsf{T} = X^\mathsf{T}(XX^\mathsf{T} + \lambda I_n)^{-1}$ (multiply both sides by suitable matrices). Conclude that the ridge solution can be written $\hat{\mathbf{w}} = X^\mathsf{T}\boldsymbol{\alpha}$ with $\boldsymbol{\alpha} = (XX^\mathsf{T} + \lambda I)^{-1}\mathbf{y}$, and compare the costs of the two forms when $d \gg n$.

D. Programming

D1. Implement Algorithms 2, 3 and 5 in numpy and compare their results and running times for $n = 10^4$, $d = 100$. Then make two columns nearly collinear ($\mathbf{f}_2 = \mathbf{f}_1 + 10^{-6}\times$noise) and compare the accuracy of the normal equations with that of QR.

D2. Reproduce Figure 2. Then fit least squares with gradient descent from $\mathbf{w} = \mathbf{0}$ and stop after k steps: compare the spread of the estimates with that of ridge. Which k behaves like $\lambda = 3$?

D3. Implement Algorithm 7. Check it on the hand calculation of Section 5.3 ($\lambda = 4$ must give $(1.5, 0)$), then reproduce the lasso panel of Figure 4 with warm starts and choose λ by 5-fold cross-validation. Optionally, recover the two terms of a simulated damped oscillator $\ddot{z} = -4z - 0.3\dot{z}$ from the library $\{1, z, \dot{z}, z^2, z\dot{z}, \dot{z}^2\}$, as in the physics box of Section 6.

Further reading. Hastie, Tibshirani and Friedman [5], chapter 3; Bishop [1], chapter 3; for sparsity, Hastie, Tibshirani and Wainwright [6].

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[2] S. L. Brunton, J. L. Proctor and J. N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, *Proc. Natl. Acad. Sci. USA* 113, 3932–3937 (2016).

[3] J. Friedman, T. Hastie and R. Tibshirani, Regularization paths for generalized linear models via coordinate descent, *J. Stat. Softw.* 33(1), 1–22 (2010).

[4] W. Gurnee and M. Tegmark, Language models represent space and time, *ICLR* (2024).

[5] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[6] T. Hastie, R. Tibshirani and M. Wainwright, *Statistical Learning with Sparsity: The Lasso and Generalizations*, CRC Press (2015).

[7] A. E. Hoerl and R. W. Kennard, Ridge regression: biased estimation for nonorthogonal problems, *Technometrics* 12, 55–67 (1970).

[8] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer (2006).

[9] R. Tibshirani, Regression shrinkage and selection via the lasso, *J. R. Stat. Soc. B* 58, 267–288 (1996).

[10] H. Zou and T. Hastie, Regularization and variable selection via the elastic net, *J. R. Stat. Soc. B* 67, 301–320 (2005).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.